

웹 서비스 및 RAG 기반 AI 솔루션

프로젝트 Wrap-Up 리포트

[26 . 06 . 30]

비스텔리전스 채용연계형 1기 2차 미니프로젝트

김지환

천승현

신동원

목차

목차.....	1
표 목차.....	2
그림 목차.....	3
1. 프로젝트 개요.....	4
1-1. 핵심 비전.....	4
1-2. 프로젝트 계획.....	4
2. 린 캔버스.....	6
2-1. 린캔버스 분석.....	6
2-2. 핵심 문제 정의 및 관점 분석 도출.....	7
2-3. 타겟 고객 및 페르소나 도출.....	7
3. 시스템 아키텍처.....	8
3-1. 시스템 물리 구조.....	8
3-2. 기술 스택.....	9
3-3. 핵심 API 라우팅 명세.....	9
4. 데이터베이스 설계 및 대용량 데이터 관리.....	11
4-1. 데이터베이스 ERD 및 관계 구조.....	11
4-2. 대용량 ArXiv 데이터셋 분석.....	12
5. 유스 케이스 정의.....	13
6. 플랫폼 기능 요구 명세서.....	15
6-1. 공통 RAG 검색 레이어 요구 명세.....	15
6-2. 일반 채팅 허브 요구 명세.....	15
6-3. 대규모 문헌 스펙 비교 및 공백 분석기 요구 명세.....	16
6-4. 맞춤형 연구 비서 썬 팩토리 요구 명세.....	16
6-5. 보안 피어 리뷰 및 디펜스 아레나 [향후 개발 로드맵 / 미구현].....	17
7. 핵심 기능별 에이전트 파이프라인 설계.....	18
7-1 일반 채팅 허브 파이프라인 플로우.....	18
7-2. 대규모 문헌 분석기 파이프라인 플로우.....	18
7-3. 맞춤형 연구 비서 썬 팩토리 파이프라인 플로우.....	19
8. 시스템 종합 시퀀스 다이어그램.....	21
8-1. 오프라인 로컬 데이터 배치 적재 파이프라인.....	21
8-2. 실시간 병렬 융합 RAG 에이전트 스트리밍.....	22
8-3. 비동기 백그라운드 배치 공백 분석.....	23
8-4. 맞춤형 연구 비서 Gem 생성.....	24
9. 정량적 성능 평가 및 품질 검증.....	26
9-1. 테스트 개요.....	26
9-2. 정량적 성능 평가.....	26
9-3. 자동화 테스트 스위트 실행 결과.....	28
9-4. 수동 검증 매트릭스.....	30
10. 핵심 기술 문제 해결 사례.....	31
11. 한계 및 향후 발전 로드맵.....	32
11-1. 현재 시스템의 기술적 한계점.....	32
11-2. 3대 핵심 향후 발전 로드맵.....	32
References.....	35

표 목차

[표 1] 팀 역할 분담
[표 2] 프로젝트 마일스톤
[표 3] 린캔버스 핵심 전략
[표 4] 핵심 문제 관점 분석
[표 5] 타겟 고객 페르소나
[표 6] 레이어별 기술 스택
[표 7] 핵심 API 라우팅 명세
[표 8] 도메인별 데이터 적재 현황
[표 9] 공통 RAG 검색 레이어 명세
[표 10] 일반 채팅 허브 기능 명세
[표 11] 공백 분석기 기능 명세
[표 12] 썬 팩토리 기능 명세
[표 13] 보안 피어 리뷰 및 디펜스 아레나 기능 명세
[표 14] 채팅 허브 파이프라인 단계
[표 15] 대규모 문헌 분석기 파이프라인 단계
[표 16] 썬 팩토리 파이프라인 단계
[표 17] 데이터 배치 적재 프로세스
[표 18] 실시간 병렬 RAG 프로세스
[표 19] 비동기 공백 분석 프로세스
[표 20] 연구 비서 Gem 생성 프로세스
[표 21] RAG 레이턴시 최적화 지표
[표 22] 자동화 테스트 스위트 실행 결과
[표 23] 수동 검증 매트릭스
[표 24] 사례 1 해결 내역
[표 25] 사례 2 해결 내역
[표 26] 사례 3 해결 내역

그림 목차

- [그림 1] 시스템 전역 오케스트레이션
- [그림 2] 시스템 물리 구조도
- [그림 3] 데이터베이스 ERD
- [그림 4] HNSW 인덱스 생성문
- [그림 5] 유스 케이스: 일반 챗 허브
- [그림 6] 유스 케이스: 연구 공백 분석기
- [그림 7] 유스 케이스: 연구 비서 Gem 팩토리
- [그림 8] 유스 케이스: 보안 피어 리뷰 및 디펜스 아레나
- [그림 9] 오프라인 로컬 데이터 배치 적재 파이프라인 시퀀스
- [그림 10] 실시간 병렬 융합 RAG 에이전트 스트리밍 시퀀스
- [그림 11] 비동기 백그라운드 배치 공백 분석 시퀀스
- [그림 12] 맞춤형 연구 비서 Gem 생성 시퀀스
- [그림 13] 응답 레이턴시 최적화 비교
- [그림 14] 향후 발전 로드맵
- [그림 15] LangGraph 기반 다중 에이전트 토론 엔진

1. 프로젝트 개요

현대 R&D 환경은 정보의 폭발적 증가와 기술적 복잡성의 심화라는 병목 현상에 직면해 있습니다. 매년 수백만 편의 논문이 쏟아지는 상황에서, 연구자가 유의미한 가설을 설정하고 검증하는 본연의 가치에 집중하기보다는 자료 수집과 정리에 과도한 인적 자원을 소모하고 있습니다. 본 프로젝트는 이러한 '연구 생산성 효율 개선'을 목표로, 단순한 검색 도구를 넘어 연구 프로세스 전체를 가속화하는 지능형 아키텍처를 구축하고자 설계되었습니다.

1-1. 핵심 비전

"연구자 및 R&D 센터를 위한 맞춤형 AI 공동 연구 파트너"

본 프로젝트는 파편화된 학술 데이터와 실시간 기술 동향을 단일 지능형 컨텍스트로 통합합니다. 대규모 문헌 분석 아키텍처와 커스텀 가능한 에이전트를 통하여, 연구자가 정보 과부하에서 벗어나 전략적 의사결정에만 전념할 수 있는 최적의 공동 연구 환경을 제공하는 것을 지향합니다.

1-2. 프로젝트 계획

본 프로젝트는 각 도메인별 전문 RAG 파이프라인 구축을 위해 다음과 같이 역할 분담을 하였으며, 6월 16일부터 30일까지의 개발 마일스톤을 통해 완성되었습니다.

1-2-1. 팀 역할 분담

담당자	담당 역할
김지환	컴퓨터 과학 분야 RAG 파이프라인 구현 및 대규모 문헌 분석 기능 개발
신동원	천문학 분야 RAG 파이프라인 구현 및 질 팩토리 기능 개발
천승현	생명공학 분야 RAG 파이프라인 구현 및 채팅 허브 기능 개발

[표 1] 팀 역할 분담

1-2-2. 프로젝트 마일스톤

날짜	마일스톤 내용
6/16 ~ 6/17	3대 핵심 분야 총 106,974건의 데이터셋 적재 완료
6/18 ~ 6/26	채팅 허브, 비동기 문헌 분석, 젼 팩토리 등 주요 서비스 구현 완료
6/29 ~ 6/30	전 과정 통합 테스트 수행 및 시스템 배포

[표 2] 프로젝트 마일스톤

2. 린 캔버스

프로젝트의 명확한 지향점을 설정하고, 시장의 실제 문제점과 해결 방안을 연계하여 최적의 플랫폼 기능 명세를 정의하고자 린 캔버스를 작성하였습니다.

2-1. 린캔버스 분석

블록	핵심 전략 내용
1. 문제	문헌 서치 피로, 학술적 환각, IP 유출 리스크, 단발성 대화의 한계
2. 고객군	석·박사 연구원, 대학교수, 기업 R&D 센터 책임연구원, 특허 변리사
3. 가치 제안	학술 데이터와 실시간 웹 정보의 병렬 융합, 비동기 대규모 문헌 분석 대시보드
4. 솔루션	듀얼 트랙 RAG 스트리밍, 연구 공백 분석기, 사용자 정의 Gem 팩토리
5. 채널	학술 커뮤니티(김박사넷 등), Github 오픈소스 배포, 기술 컨퍼런스 브리핑
6. 수익원	Pro 멤버십(\$19.99/월), 기업용 보안 패키지 및 On-premise 구축 라이선스
7. 비용 구조	LLM API 사용료, pgvector RDS 인프라, 실시간 검색 API(Tavily 등)
8. 핵심 지표	세션당 대화 깊이, 연구 공백 도출 성공률, 인용 출처 CTR
9. 경쟁 우위	pgvector HNSW 고속 검색 아키텍처, 100% 팩트 보존 번역 필터, 보안 샌드박스 로드맵

[표 3] 린캔버스 핵심 전략

2-2. 핵심 문제 정의 및 관점 분석 도출

핵심 문제	문제 내용
	관점 분석
시간 손실	선행 연구 분류표 작성에 주당 20시간 이상 소요.
	연구자의 고임금 가치가 창의적 분석이 아닌 단순 문서 작업에 소진되어 생산성 저하
학술적 환각	AI의 거짓 정보로 인한 원문 재검증 비용 발생.
	AI에 대한 불신으로 인해 결국 기존 방식의 수동 검색으로 회귀하여 도입 효과 상쇄
데이터 보안 리스크	미발표 논문 초안이나 핵심 기밀 데이터의 외부 유출 우려.
	수십억 원 가치의 특허권 및 독점 기술 확보 기회 상실이라는 치명적 경제적 손실 초래
단발성 대화의 한계	복잡한 문헌 비교를 위해 여러 대화창을 오가며 발생하는 인지적 파편화.
	맥락 유지 실패로 인해 심층적인 논리적 추론 및 통찰 도출 불가

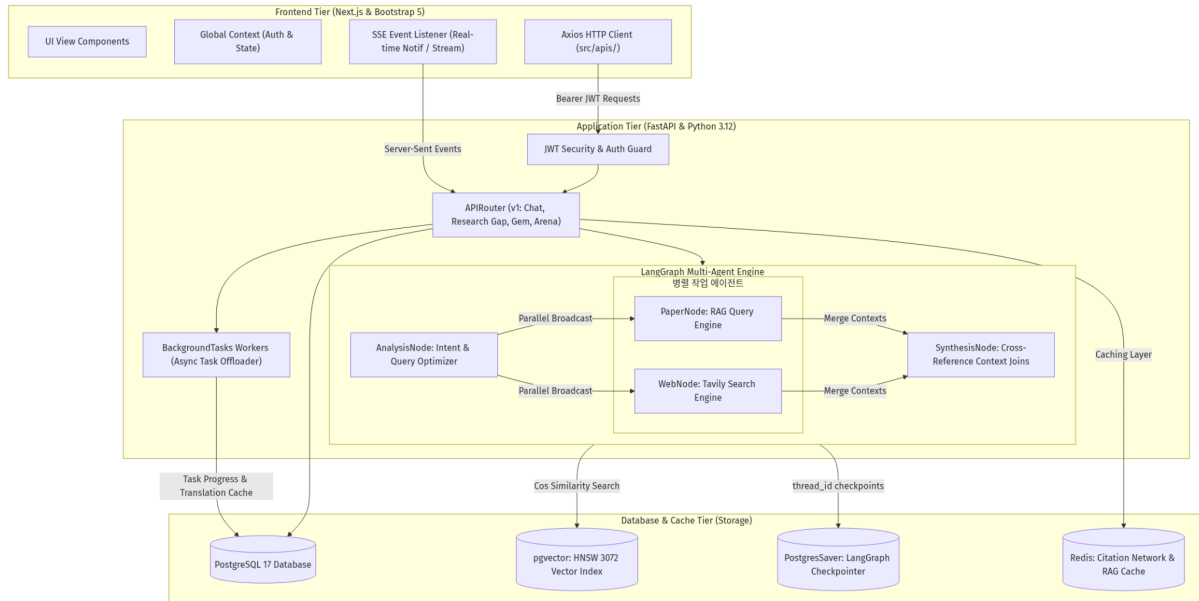
[표 4] 핵심 문제 관점 분석

2-3. 타겟 고객 및 페르소나 도출

고객	페르소나
박지수(26, 석사과정)	최신 NLP 트렌드 파악과 학술 팩트 확인 사이의 시간 압박을 해결하고자 하며, 병렬 RAG 챗을 통해 논문 원리와 상용화 적용 현황을 동시 분석합니다.
박영지(31, 비스텔리전스)	새로운 연구 주제를 찾기 위한 문헌 조사 공수를 줄이고자 하며, 연구 공백 분석기를 통해 수십 편의 논문에서 미개척 영역을 단숨에 도출합니다.

[표 5] 타겟 고객 페르소나

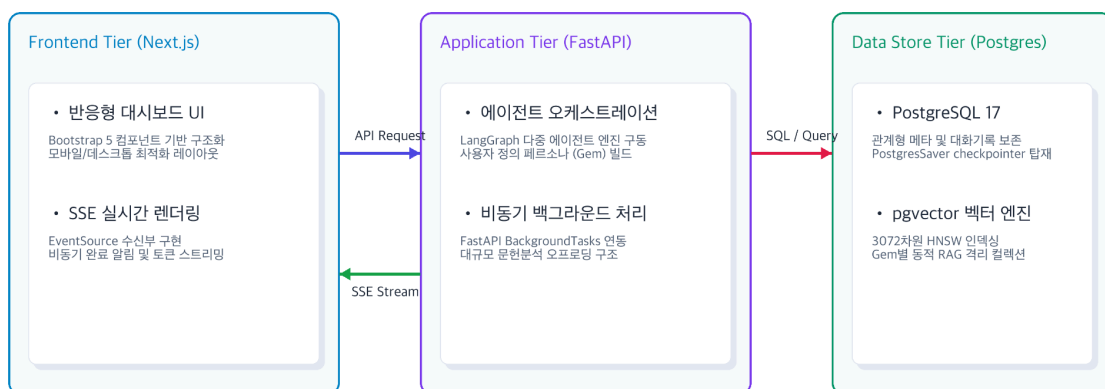
3. 시스템 아키텍처



[그림 1] 시스템 전역 오케스트레이션

본 프로젝트는 데이터 격리와 시스템 확장성을 보장하기 위해 정교하게 설계되었습니다. 대규모 데이터 처리와 안정적인 상태 관리를 최우선으로 고려한 물리적 아키텍처와 지능형 에이전트 워크플로우로 구성됩니다.

3-1. 시스템 물리 구조



[그림 2] 시스템 물리 구조도

프론트엔드, 애플리케이션, 데이터 레이어를 물리적으로 분리하여 보안성과 유지보수 효율을 극대화했습니다.

- **Frontend:** Next.js (반응형 대시보드 및 SSE 실시간 렌더링)
- **Application:** FastAPI (에이전트 오케스트레이션 및 비동기 처리)
- **Data Store:** PostgreSQL 17 + pgvector (HNSW 인덱싱 기반 벡터 엔진)

3-2. 기술 스택

레이어	기술 스택	역할 및 용도	주요 선정 이유 및 특징
Frontend	• Next.js (React) • JavaScript (ES6+)	• 사용자 웹 대시보드 인터페이스 구축 • 비동기 클라이언트 통신 모듈 • 반응형 그리드 및 UI 컴포넌트	• App Router 기반의 고효율 서버/클라이언트 컴포넌트 분리 • Bootstrap CSS를 활용한 인라인 스타일 배제 및 UI 일관성 확보 • Axios 인터셉터를 통한 JWT 토큰 관리
Backend	• FastAPI • Python 3.12 • Pydantic v2	• 비동기 API 서버 엔진 • 데이터 검증 및 DTO(Data Transfer Object) 통제	• Asynchronous ASGI 기반의 고속 Q&A 스트리밍 및 SSE 알림 브로드캐스팅 지원 • Pydantic BaseDTO 상속을 통한 REST API 스키마 정합성 보장
Database & Storage	• PostgreSQL 17 • pgvector 0.7+ • SQLAlchemy 2.0+	• 관계형 데이터 영구 적재 • 3대 학술 도메인 10만 건 임베딩 관리 • 비동기 DB 커넥션 ORM 매핑 • 인메모리 캐싱 및 실시간 Pub/Sub	• HNSW(Hierarchical Navigable Small World) 인덱스 탑재로 코사인 유사도 검색 속도 최적화 (68ms) • LangGraph의 AsyncPostgresSaver 연동을 통해 스레드 체킹 히스토리 백업
AI Agent	• LangGraph • LangChain • OpenAI API • Tavily Search API	• 멀티 에이전트 상태 전이 및 워크플로우 • 듀얼 트랙 병렬 LLM 오케스트레이션 • 실시간 외부 시사/오픈소스 정보 수집	• asyncio.gather를 통한 paper_node와 web_node 병렬 가동 구조 제어 • gpt-4o-mini/gpt-4o 결합을 통한 비용 최적화 답변 합성 및 번역 수행
Testing & Infrastructure	• Docker & Compose • Pytest • NPM / Pip	• 로컬 통합 컨테이너 환경 가동 • 비동기 API 단위 테스트 수행 • 패키지 의존성 관리	• docker-compose 단일 명령을 통한 FE/BE/DB 전체 로컬 구동 표준화 • 테스트 시 외부 API 종속 제거를 위한 Mocking 수트 완성

[표 6] 레이어별 기술 스택

3-3. 핵심 API 라우팅 명세

도메인	메서드	API 경로	설명	Request DTO	Response DTO
인증	POST	/api/v1/auth/login	Swagger UI Authorize 연동 토큰 발급	username, password (Form)	{ "access_token": "...", "token_type": "bearer" }
대화	POST	/api/v1/chat/sessions	신규 대화 세션방 생성	{ "title": "세션방 제목" }	{ "session_id": "UUID", "title": "... " }

[웹 서비스 및 RAG 기반 AI 솔루션 프로젝트]

도메인	메서드	API 경로	설명	Request DTO	Response DTO
대화	POST	/api/v1/chat/sessions/{session_id}/messages/multi/stream	멀티 에이전트 RAG & Web 용합 SSE 스트리밍	{ "message": "질문", "image": "Base64 (옵션)" }	text/event-stream (token, status, route, source)
대화	GET	/api/v1/chat/sessions/{session_id}/messages	대화 세션 내 메시지 히스토리 및 출처 전체 조회	없음	{ "messages": [{ "role": "human", "content": "... " }] }
분석	POST	/api/v1/research-gap/analyze	비동기 대규모 문헌 비교 분석 예약 생성	{ "domain": "cs", "query": "Transformer" }	{ "task_id": "UUID" }
분석	GET	/api/v1/research-gap/tasks/{task_id}	비동기 분석 진행 상태 및 진행률(%) 조회	없음	{ "task_id": "UUID", "status": "RUNNING", "progress": 40 }
분석	GET	/api/v1/research-gap/tasks/{task_id}/result	분석 완료된 최종 매트릭스 및 공백 리포트 조회	없음	{ "result": { "matrix": [...], "report": "... " } }
분석	POST	/api/v1/research-gap/tasks/{task_id}/translate	분석 결과 한글 번역 및 인용구 원어 백업/복원	없음	번역 처리된 매트릭스 및 리포트 JSON
Gem	POST	/api/v1/gems	커스텀 연구 비서 Gem 인스턴스 설정 생성	{ "name": "바이오비서", "db_sources": ["bio"] }	{ "gem_id": "UUID", "name": "바이오비서", ... }
Gem	POST	/api/v1/gems/{gem_id}/files	개인 문헌 PDF 업로드 및 테넌시 격리 임베딩	files: UploadFile (Multipart)	{ "file_count": 1, "chunk_count": 24 }
Gem	POST	/api/v1/gems/{gem_id}/chat/stream	젬 전용 벡터 컬렉션 참조 격리 대화 스트리밍	{ "thread_id": "UUID", "message": "질문" }	text/plain 실시간 캐릭터 토큰 스트림
알림	GET	/api/v1/notification/stream	실시간 SSE 알림 스트림 연결 수립	없음	text/event-stream (Realtime Notification Event)

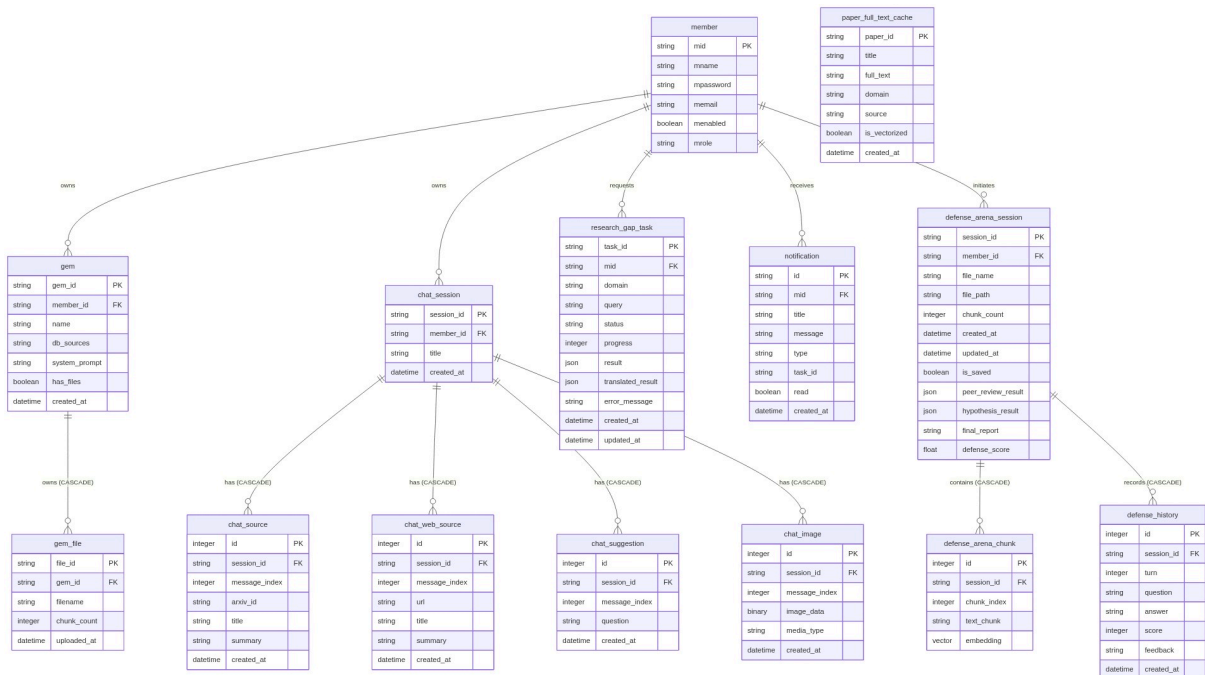
[표 7] 핵심 API 라우팅 명세

4. 데이터베이스 설계 및 대용량 데이터 관리

본 플랫폼은 고성능 학술 RAG 검색의 신뢰성을 보장하기 위해 관계형 메타데이터의 영속성과 고차원 벡터 임베딩 검색의 효율성을 결합한 하이브리드 데이터베이스 아키텍처를 채택했습니다.

4-1. 데이터베이스 ERD 및 관계 구조

시스템의 핵심 관계형 테이블(회원, 채팅 세션, 인용 출처, 특허 비서 켄, 비동기 분석 작업)과 LangChain pgvector 표준 임베딩 스키마의 상호 참조 관계를 구조화한 물리 ERD 다이어그램입니다.



[그림 3] 데이터베이스 ERD

4-1-1. LangChain PGVector 표준 규격 수용

- 3대 학술 분야의 임베딩 데이터는 collection 및 embedding 테이블로 이원화 관리합니다.
- 각 도메인(cs, bio, astronomy) 및 사용자 정의 Gem 파일 컬렉션들은 collection_id 외래키를 통해 물리적으로 격리되어 RAG 수행 시 불필요한 스캔 범위를 최소화합니다.

4-1-2. HNSW (Hierarchical Navigable Small World) 인덱스 적용

- 3072차원의 초고차원 벡터 검색 속도를 비약적으로 향상시키고 인덱스 탐색 성능을 보장하기 위해, 코사인 유사도 연산 기반 HNSW 인덱스를 생성했습니다.
- 물리 DDL 설정:

```
CREATE INDEX idx_langchain_pg_embedding_hnsw
ON langchain_pg_embedding USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

[그림 4] HNSW 인덱스 생성문

4-2. 대용량 ArXiv 데이터셋 분석

본 플랫폼은 고가치 학술 도메인 지식을 정밀 수용하고자, 약 200만 편 이상의 학술 논문 메타데이터를 포함하는 Kaggle의 ArXiv Dataset을 분석 및 가공하여 RAG 인프라의 기초 지식으로 통합했습니다.

4-2-1. 3대 타겟 도메인 카테고리 필터링 및 적재 현황

전체 데이터셋 중 플랫폼 비즈니스 목적에 부합하는 IT 신기술, 천문과학, 백신 및 유전체 분야에 대응하는 고유 학술 카테고리를 추출하여 총 106,974건의 대용량 논문 데이터를 최종 데이터베이스에 안착시켰습니다.

학술 도메인 구분	최종 적재 건수	적재 상세 범위
컴퓨터 과학	17,825 건	cs.NE 전체 범위
천문학	35,083 건	astro-ph.EP 전체 범위
생명과학	54,066 건	q-bio.GN 및 주요 유전공학 서브 카테고리 일체
합계	106,974 건	3대 도메인 타겟 핵심 학술 문헌 전체 적재 완료

[표 8] 도메인별 데이터 적재 현황

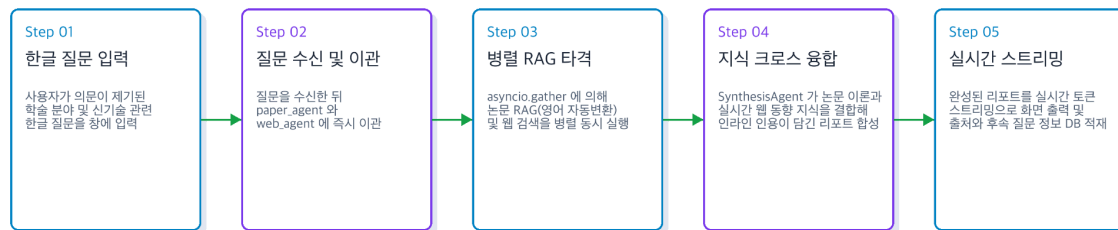
4-2-2. RAG 데이터 전처리 및 텍스트 청킹 규칙

- 지식 구성: RAG 파이프라인의 유사도 비교 정확도를 극대화하고 검색 텍스트 분절 시 발생하는 문맥 손실을 원천 방어하기 위해, 논문의 **제목**과 **초록**을 결합하여 단일 문서로 적재했습니다.
- 단일 벡터 청킹 정책: 학술 정보의 엄밀함을 보존하기 위해 논문 초록은 인위적인 중도 분할을 거치지 않고, 각 논문의 초록 전체를 단일 임베딩 벡터로 일괄 업로드하여 1:1 대응 관리를 수행합니다.
- 임베딩 벡터 명세: OpenAI의 최신 임베딩 모델인 text-embedding-3-large를 적용하여 3,072차원의 고밀도 벡터 데이터를 생성하고 pgvector 물리 레코드에 매핑하였습니다.

5. 유스 케이스 정의

각 기능은 연구자의 인지적 부하를 최소화하고 데이터 기반의 통찰력을 즉각적으로 제공하는 데 초점을 맞추고 있습니다.

[UC-01] 일반 챗 허브



[그림 5] 일반 챗 허브 파이프라인

사용자의 학술적 질문에 대해 별도의 라우팅 분기 판단대기 없이, paper_agent와 web_agent를 즉시 비동기 병렬로 동시 실행합니다. 질문의 번역 및 학술 용어 추출은 paper_agent 내부에서 자체 쿼리 변환 지침에 따라 자율적으로 수행하여 신뢰성 높은 학술 이론과 최신 시장 동향이 교차 융합된 마크다운 리포트를 실시간 토큰 단위로 스트리밍합니다. 이는 연구자가 정보를 수집하고 합성하는 과정의 인지적 단계를 단일 과정으로 통합합니다.

[UC-02] 연구 공백 분석기



[그림 6] 연구 공백 분석기 파이프라인

수십 편의 선행 연구를 배치로 처리하여 각 논문의 문제점과 한계점을 분석하는 매트릭스를 자동 생성합니다. 특히 학술 번역 시 팩트 보존 메커니즘을 적용하여, 한국어 번역 과정에서도 핵심 영문 인용구는 변형 없이 원어를 유지함으로써 검증의 정확성을 보장합니다. 연구자는 이를 통해 며칠이 소요되던 'Research Gap' 도출 작업을 수분 만에 완료할 수 있습니다.

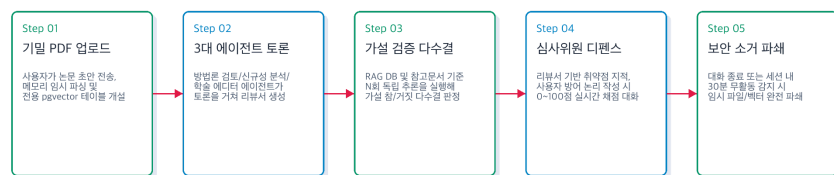
[UC-03] 연구 비서 Gem 팩토리



[그림 7] Gem 팩토리 파이프라인

사용자는 자신의 특정 연구 도메인과 페르소나를 조합하여 독립된 에이전트(Gem)를 생성할 수 있습니다. 각 Gem은 격리된 전용 pgvector 컬렉션을 운영하여 개인 연구 문서가 타 에이전트나 사용자에게 유출되지 않도록 기술적으로 격리합니다. 이는 맞춤형 R&D 비서를 즉각적으로 고용하는 것과 같은 효과를 제공합니다.

[UC-04] 보안 피어 리뷰 및 디펜스 아레나 (향후 개발 로드맵 - 미구현)



[그림 8] 유스 케이스: 보안 피어 리뷰 및 디펜스 아레나

외부 유출이 절대 안 되는 기밀 연구용 PDF 문서를 임시로 올려 다중 에이전트의 피드백을 받고, 가상의 학회 심사위원 에이전트와 모의 질의응답을 거쳐 논리적 허점을 철저히 자가 디펜스 및 평가합니다.

6. 플랫폼 기능 요구 명세서

6-1. 공통 RAG 검색 레이어 요구 명세

모든 분석 및 대화 기능군이 공유하는 백엔드 공통 RAG 검색 엔진(pgvector 기반)의 도메인별 요약 테이블 및 3대 상세 기능 요구 명세입니다. 모든 기능군은 백엔드 공통 RAG 검색 레이어를 공유하며, HNSW인덱스를 탑재하여 코사인 거리 0.4 이하의 신뢰성 높은 청크를 고속 추출합니다.

코드	특화 도메인	데이터 입출력 규격	기술 구현 상세
F-RAG-01	생명공학	In: query, top_k=10 Out: doc_id, title, text, score	유전자 편집 및 단백질 분석 최적화. 3072차원 고차원 임베딩 매칭.
F-RAG-02	컴퓨터 과학	In: query, top_k=4 Out: doc_id, title, text, score	신경망 진화 및 딥러닝 핵심 개념 탐색을 위한 고속 엔진 구축.
F-RAG-03	천문학	In: query, top_k=10 Out: doc_id, title, text, score	외계행성 및 궤도역학 이론 검증용 대형 임베딩 공간 확보.

[표 9] 공통 RAG 검색 레이어 명세

6-2. 일반 채팅 허브 요구 명세

일반 채팅 허브는 자연어 질문에 대해 논문 RAG와 웹 실시간 검색을 무조건적으로 병렬 가동하여, 융합 지식을 단일 마크다운 답변으로 완성하고 실시간 토큰 스트리밍을 공급하는 기본 제어 통제판입니다.

코드	상세 기능명	비즈니스 규칙 및 작동 로직
F-01-01	병렬 RAG 및 웹 융합 검색	• 라우팅 분기를 거치지 않고, asyncio.gather 비동기 라이브러리를 통해 논문 검색과 웹 검색을 무조건 병렬로 동시 실행하여 I/O 병목을 제거하고 응답 속도를 보장 단축함.
F-01-02	SSE 실시간 스트리밍 답변	• GPT-4o 최종 마크다운 합성 답변을 HTTP Server-Sent Events 방식을 사용해 토큰 발생 시마다 실시간 푸시 및 타자 효과 렌더링.
F-01-03	인용 출처 매핑 및 적재	• 대화 RAG 컨텍스트로 실제 참조된 ArXiv 논문 서지 메타데이터(ID, 제목)를 답변과 인덱스 매핑하여 대화 완료 즉시 영구 적재.

[표 10] 일반 채팅 허브 기능 명세

6-3. 대규모 문헌 스펙 비교 및 공백 분석기 요구 명세

대규모 문헌 명세 비교 및 공백 분석기는 수십 편의 선행 연구 데이터를 비동기 배치로 분석하여 '해결 과제'와 '한계점'을 도출하는 독립 대시보드입니다.

코드	상세 기능명	비즈니스 규칙 및 작동 로직
F-02-01	비동기 배치 분석 접수	• 분석 요청 즉시 고유 task_id를 발급하고 데이터베이스 상태를 PENDING으로 적재하며 연산은 BackgroundTasks로 오프로딩함.
F-02-02	학술 번역 및 원어 복원	• 영문 분석 결과를 한국어로 번역하되, 오역 및 팩트 왜곡 방지를 위해 핵심 인용구 필드는 오리지널 영문으로 강제 오버라이트 보존.
F-02-03	SSE 완료 알림 및 캐싱	• 진행률 100% 도달 즉시 DB status를 COMPLETED로 변경하고, notification 테이블 적재 및 SSE를 통해 실시간 완료 토스트 푸시.

[표 11] 공백 분석기 기능 명세

6-4. 맞춤형 연구 비서 젼 팩토리 요구 명세

맞춤형 연구 비서 젼 팩토리는 사용자가 특정 RAG 소스 참조 카테고리 및 시스템 프롬프트 지침을 조합하여 개인화된 특화 비서를 개설하고 대화를 수행하는 기능입니다.

코드	상세 기능명	비즈니스 규칙 및 작동 로직
F-03-01	맞춤형 페르소나 Gem 개설	• 사용자가 설정한 RAG 참고 분야(cs, bio, astronomy) 및 시스템 프롬프트 가이드라인을 바인딩하여 젼 메타데이터 생성.
F-03-02	젼 전용 파일 적재 및 격리	• 개인 PDF 파일을 500자 청크 분할 및 임베딩하여 gem_{gem_id}_files 전용 컬렉션에 적재하고, 타 젼과의 RAG 데이터 접근을 물리적 격리.
F-03-03	젼 삭제 및 데이터 영구 소멸	• 젼 삭제 요청 시, Cascade 규칙에 의해 관련 대화 메타를 삭제하고, gem_{gem_id}_files 컬렉션을 DB에서 물리적으로 드롭하여 완전 소거.

[표 12] 젼 팩토리 기능 명세

6-5. 보안 피어 리뷰 및 디펜스 아레나 [향후 개발 로드맵 / 미구현]

코드	상세 기능명	비즈니스 규칙 및 작동 로직
F-04-01	PDF 격리 업로드	• 연구자별 격리 디렉토리에 문서를 업로드하고, 해당 세션 전용 임시 pgvector 컬렉션을 동적으로 생성하여 타 세션의 접근을 원천 차단함.
F-04-02	30분 수명 주기 소거	• 30분 무활동 감지 시 백그라운드 Shredding 데몬이 작동하여, PDF 원본 파일 및 할당된 임시 벡터 인덱스를 물리적으로 완전 파쇄함.
F-04-03	다중 에이전트 토론	• 방법론 검증, 신규성 분석, 학술 에디터 3인 에이전트가 상태(State)를 공유하며 토론을 벌여, 종합 피드백 DTO를 도출하는 구조.
F-04-04	자기 일관성 가설 검증	• N회 독립 추론을 통한 다수결 합의(Self-Consistency) 알고리즘을 적용하여, 제안된 가설의 참(SUPPORT) 또는 거짓(REFUTE)을 판정함.
F-04-05	압박 질문 디펜스 챗	• 심사위원 에이전트가 연구의 한계점을 질문하고, 이에 대한 사용자의 방어 답변을 실시간 채점하여 점수와 피드백을 제공하는 대화형 시뮬레이션.

[표 13] 보안 피어 리뷰 및 디펜스 아레나 기능 명세

7. 핵심 기능별 에이전트 파이프라인 설계

7-1 일반 채팅 허브 파이프라인 플로우

일반 채팅 허브는 질문의 라우팅 분기 판단에 따른 대기 지연을 예방하고, 학술 논문 DB와 실시간 웹 정보를 무조건 병렬로 인출 및 융합 합성하는 단방향 실시간 파이프라인입니다.

단계	내용
Step 1. 질의 진입 및 쿼리 이관	사용자의 입력 질문이 접수되면 라우팅 분석기를 거치지 않고, 수신된 질문 텍스트를 paper_agent와 web_agent에 즉시 이관합니다.
Step 2. 듀얼 트랙 무조건적 병렬 인출	조건부 라우팅을 거치지 않고 asyncio.gather 비동기 라이브러리를 가동합니다. paper_agent는 내부 시스템 지침에 따라 한글 질문에서 영어 학술 검색어를 자율 추출/번역해 pgvector RAG를 구동하고, web_agent는 Tavily 검색을 구동해 병렬 동시 처리합니다. 두 I/O 연산을 병렬 실행함으로써 순차 처리 대비 평균 23%의 응답 속도 향상 을 달성했습니다.
Step 3. 교차 융합 합성 및 인용 매핑	SynthesisNode가 가동되어, 검색된 고정 학술 정보(RAG)와 최신 웹 뉴스 컨텍스트를 한 프레임워크 내에서 Join합니다. 인용된 ArXiv 논문의 식별 코드를 찾아내어 답변의 근거 자료로 1:1 결합을 진행합니다.
Step 4. 스트리밍 송출 및 사후 데이터 적재	FastAPI의 StreamingResponse 비동기 제너레이터를 호출하여 답변 마크다운을 HTTP SSE 채널로 즉시 밀어냅니다. 스트리밍이 완결되는 시점에 사용된 논문 서지를 관계형 DB의 chat_source 테이블에 비동기 트랜잭션 적재하고 후속 질문을 추천 생성합니다.

[표 14] 채팅 허브 파이프라인 단계

7-2. 대규모 문헌 분석기 파이프라인 플로우

대규모 문헌 분석기는 오랜 연산 시간이 소요되는 배치 성격의 의뢰를 백그라운드 스레드에서 안정적으로 병렬 해체 및 합성하고 실시간으로 알림을 중재하는 비동기 파이프라인입니다.

단계	내용
Step 1. 분석 요청 접수 및 비동기 큐 위임	사용자가 분석 요청을 인가하면 백엔드는 고유 task_id 를 발급하고 research_gap_task 테이블에 초기 상태를 삽입합니다.

단계	내용
	사용자의 대기 차단을 막기 위해 연산 핸들러를 FastAPI BackgroundTasks 비동기 큐에 즉각 위임하고 클라이언트에 200 OK 응답을 즉시 반환합니다.
Step 2. 중복 문헌 필터링 및 유사도 상위 논문 선출	백그라운드 스레드에서 해당 도메인 pgvector 검색(k=25)을 거쳐 중복된 문헌을 배제한 뒤 가장 무관 노이즈가 없는 핵심 4개 논문의 원본 초록 본문을 안전하게 추출합니다 (진행률 40% 상태 갱신).
Step 3. 개별 문헌 팩트 해체 및 검증 근거 가드	gpt-4o-mini 모델의 Structured Output 기능을 기용하여 각 개별 논문의 핵심 해결 과제와 한계점을 엄격한 JSON DTO로 추출합니다. 이때 오역 방지와 학술적 엄밀성을 위해 팩트 검증 문장인 source_quote 문자열 리스트를 파이썬 서비스 레이어 가드를 통해 영어 원어 상태 그대로 100% 영구 보존합니다 (진행률 80% 상태 갱신).
Step 4. 공백 추론 합성 및 SSE 완료 브로드캐스트	4개 논문 limitations의 공통 교집합을 추론하고 혁신 미래 연구 과제 3선을 도출하는 종합 보고서를 최종 완성합니다. 보고서 한글 번역 객체를 DB translated_result JSONB 캐시 영역에 적재하고, SSE Broadcaster 채널을 노크하여 클라이언트 브라우저 알림창으로 "분석 완료" 이벤트를 실시간 발송합니다 (진행률 100% 완료).

[표 15] 대규모 문헌 분석기 파이프라인 단계

7-3. 맞춤형 연구 비서 젼 팩토리 파이프라인 플로우

젠 팩토리는 사용자의 맞춤형 페르소나 지침과 외부 연구 문서를 런타임에 동적으로 주입하여, 고도의 데이터 기밀 격리와 Cascade 생명주기 제어를 달성하는 파이프라인입니다.

단계	내용
Step 1. 페르소나 바인딩 및 젼 등록	사용자가 맞춤형 연구 지침 프롬프트와 참조 도메인 카테고리를 설정하면, 백엔드는 관계형 DB의 gem 테이블에 메타 레코드를 영구 저장합니다.
Step 2. 업로드 파일 온디맨드 벡터화 및 컬렉션 격리	사용자가 특정 젼에 개인 논문(PDF 등)을 업로드하면, 백그라운드에서 pdfium 파서를 통해 문서를 500자 크기로 자동 청킹합니다. text-embedding-3-large API를 통해 각 청크를 3072차원으로 변환한 뒤, 독립 생성된 전용 pgvector 저장 컬렉션인 gem_{gem_id}_files 테이블 공간에만 안전하게 벌크 적재하여 타 대화방과의 데이터 공유를 완벽하게 물리 차단합니다.

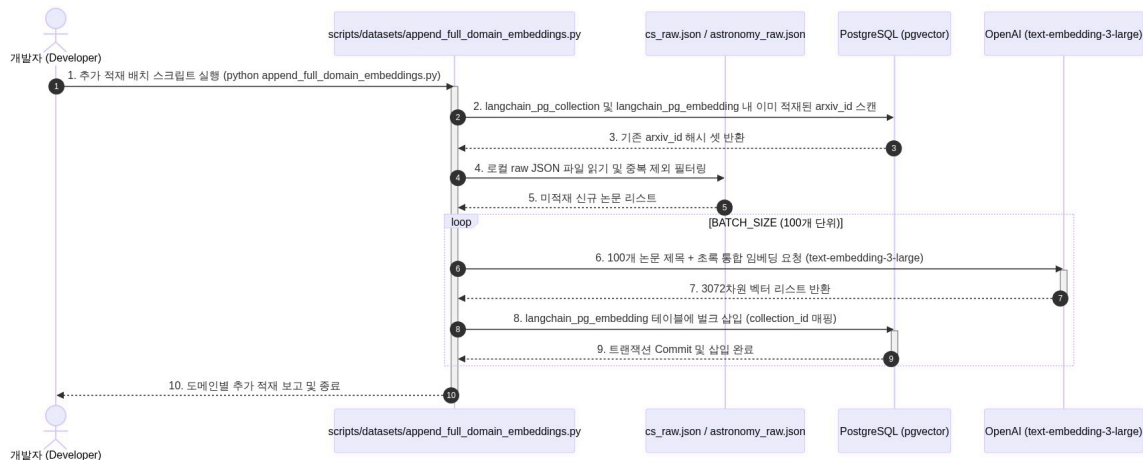
단계	내용
Step 3. 런타임 클로저(Closure) 기반 동적 도구 주입	해당 점과의 세션 대화방이 개설되면, 백엔드는 런타임 시점에 해당 gem_id를 렉시컬 스코프 내에 가두는 클로저(Closure) 함수를 동적으로 빌드합니다. 이 함수를 Supervisor Agent의 실행 컨텍스트에 톨로 실시간 바인딩하여, 에이전트가 다른 스토리지 공간을 오염시키지 않고 지정된 전용 벡터 컬렉션에서만 검색을 하도록 강제 제어합니다.
Step 4. Cascade 기반 물리 완전 소멸	사용자가 점을 삭제하면 관계형 데이터베이스 Cascade 연쇄 삭제 제약 조건에 의해 메타데이터가 영구 드롭됩니다. 이와 동시에 백엔드는 데이터베이스 연결 드라이버를 통해 gem_{gem_id}_files 컬렉션 테이블 공간을 물리적으로 DROP TABLE 처리하여 디스크 내의 모든 벡터 데이터를 흔적 없이 영구 소멸(0 Byte)시킵니다.

[표 16] 점 팩토리 파이프라인 단계

8. 시스템 종합 시퀀스 다이어그램

8-1. 오프라인 로컬 데이터 배치 적재 파이프라인

ArXiv 전체 스냅샷 및 개별 raw JSON 파일에서 도메인별 미적재 데이터를 추출하여 langchain_postgres pgvector 벌크 인덱싱을 진행하는 컴포넌트 간 상세 로직입니다.



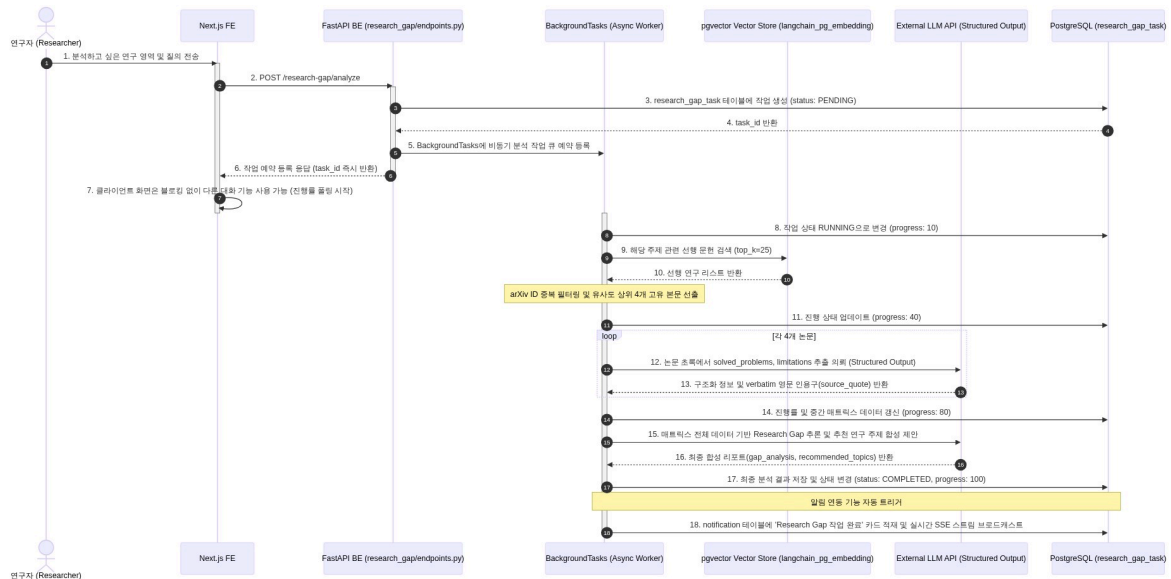
[그림 9] 오프라인 로컬 데이터 배치 적재 파이프라인 시퀀스

단계	프로세스명	상세 설명
1	배치 스크립트 기동 및 커넥션 수립	CLI를 통해 배치 스크립트 엔진을 실행하며, PostgreSQL DB 세션 및 langchain_postgres 드라이버 초기화
2	기적재 데이터 검사 및 오버랩 가드	컬렉션 ID 식별 후, 기적재된 논문(arxiv_id)을 해시셋에 메모리 로드하여 중복 방지 필터링
3	로컬 데이터 파싱 및 필터링	로컬 JSON 파일 스냅샷을 읽고, 해시셋과 대조하여 미적재된 신규 데이터만 Target Ingestion Queue에 저장
4	LLM 벌크 임베딩 요청 및 응답	100건 단위(Chunking)로 분할하여 OpenAI text-embedding-3-large API를 통해 3072차원 벡터 획득
5	PGVector 벌크 삽입 및 트랜잭션	획득한 벡터와 메타데이터를 테이블에 벌크 INSERT하고, 트랜잭션 Commit 후 메모리 해제
6	인덱싱 갱신 및 완료 리포트	PostgreSQL 내부 HNSW 인덱스 재빌드를 통한 검색 속도 보장 및 배치 가동 종료

[표 17] 데이터 배치 적재 프로세스

8-2. 실시간 병렬 융합 RAG 에이전트 스트리밍

사용자의 학술 질문에 대해 최적화 쿼리를 생성하고, pgvector HNSW RAG와 Tavily 실시간 검색 노드를 무조건 병렬로 동시 수행하여, 최종 합성 노드에서 교차 융합된 답변을 SSE로 스트리밍하는 세부 흐름입니다.



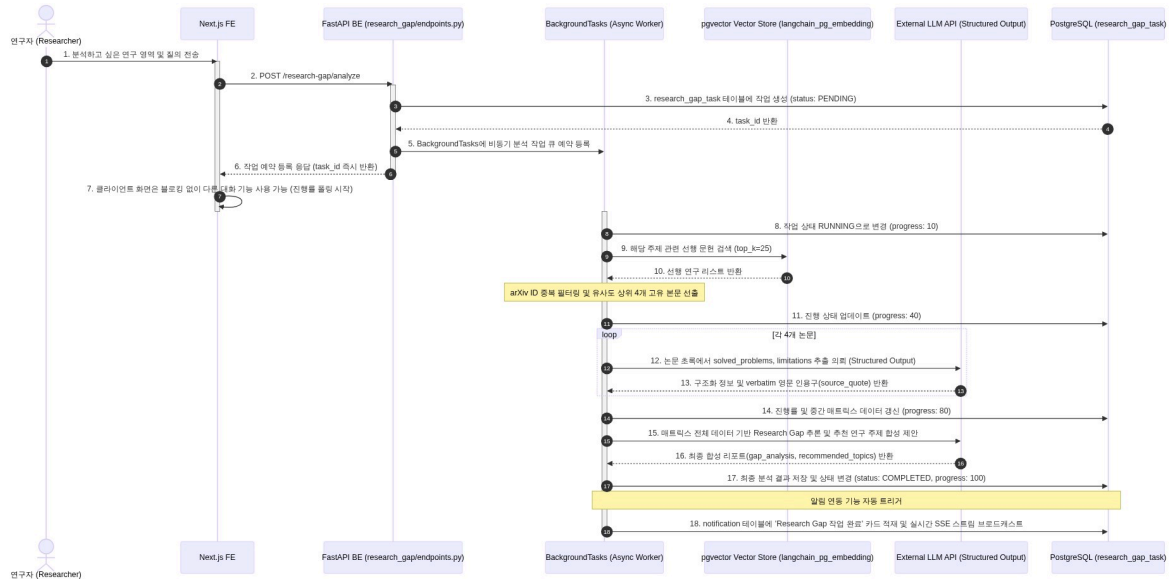
[그림 10] 실시간 병렬 융합 RAG 에이전트 스트리밍 시퀀스

단계	프로세스명	상세 설명
1	사용자 대화 요청 및 API 진입	Next.js FE에서 세션 기반 대화 메시지 스트림 API 호출 및 요청 DTO 빌드
2	쿼리 수신 및 작업 위임	ChatMultiAgentSupervisor가 질문을 수신하여 라우터 분기없이 즉시 paper_agent와 web_agent로 전달
3	비동기 병렬 RAG 동시 타격	asyncio.gather를 통해 DB 검색과 웹 검색을 병렬 수행하여 I/O 병목 해소
4	합성 엔진 구동 및 스트리밍	gpt-4o 기반 컨텍스트 합성 및 FastAPI StreamingResponse를 통한 토큰 단위 SSE 실시간 전송
5	사후 메타데이터 영구 적재	대화 종료 후 인용 논문 정보를 chat_source 테이블 저장 및 추천 질문 자동 생성
6	컨텍스트 리바인딩 및 화면 완성	스트림 종료 감지 후, 대화 이력 API를 호출하여 최종 출처 카드 및 추천 질문 UI 바인딩

[표 18] 실시간 병렬 RAG 프로세스

8-3. 비동기 백그라운드 배치 공백 분석

대규모 선행 연구에 대한 배치 분석 요청을 예약하면 백엔드 BackgroundTasks가 비동기로 가동되어 한계점을 일괄 취합하고 알림 인박스에 저장하는 흐름입니다.



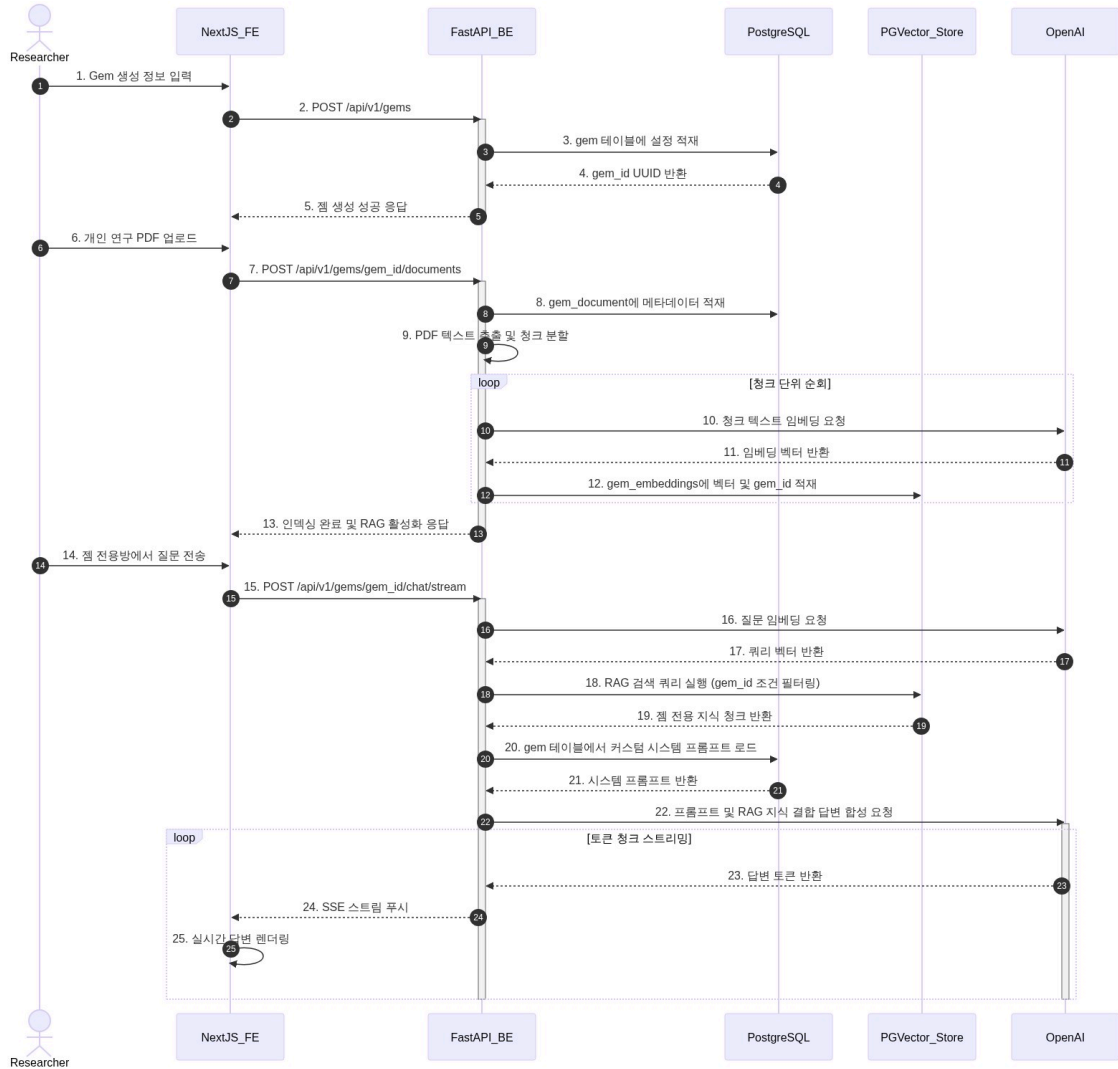
[그림 11] 비동기 백그라운드 배치 공백 분석 시퀀스

단계	프로세스명	상세 설명
1	배치 분석 요청 접수	요청 시 작업 ID 즉시 발급 및 PENDING 상태 저장, 비동기 워커 예약 후 즉시 제어권 반환(Polling 시작)
2	워커 작동 및 RAG 문헌 검색	RAG 컬렉션에서 유사도 상위 논문 25개 쿼리 후 고유 핵심 논문 4개 선별 및 상태 업데이트(10%~40%)
3	구조화 정보 추출 및 Safe Overwrite	Structured Output API로 핵심 문제 및 한계점 JSON 추출, 인용구 원어 보존 가드 필터링 적용(40%~80%)
4	연구 공백 추론 및 가설 제안	공통 공백 분석 및 후속 연구 방향성합성으로 최종 분석 매트릭스 완성(80%~100%)
5	작업 종료 및 알림 브로드캐스트	분석 결과 영구 저장 및 COMPLETED 상태 변경, SSE 스트림을 통한 작업 완료 이벤트 실시간 발송
6	화면 갱신 및 토스트 노출	SSE 이벤트 수신 시 UI 우측 상단 실시간 완료 토스트 팝업 및 인박스 카운트 갱신

[표 19] 비동기 공백 분석 프로세스

8-4. 맞춤형 연구 비서 Gem 생성

맞춤형 연구 비서 Gem은 사용자의 페르소나 지침과 개인 문서를 결합하여, 독립된 데이터 공간에서 보안이 강화된 RAG 서비스를 제공하기 위한 생성 및 관리 시퀀스를 따릅니다.



[그림 12] 맞춤형 연구 비서 Gem 생성 시퀀스

단계	프로세스명	상세 설명
1	연구 비서 Gem 인스턴스화	사용자 요청에 따른 Gem 메타데이터 저장 및 고유 gem_id 발급
2	개인 문서 격리 인덱싱	PDF 파싱 및 500자 청크 분할, gem_id 기반 물리 격리 컬렉션 적재로 타 데이터와의 교차 오염 방지

[웹 서비스 및 RAG 기반 AI 솔루션 프로젝트]

단계	프로세스명	상세 설명
3	젼 전용 RAG 조회	WHERE gem_id 조건 바인딩을 통해 지정된 젼의 전용 개인 컨텍스트 내에서만 유사도 검색 수행
4	페르소나 결합 및 스트리밍	고유 시스템 프롬프트(Persona)와 RAG 컨텍스트를 합성하여 최종 맞춤형 답변 실시간 SSE 스트리밍

[표 20] 연구 비서 Gem 생성 프로세스

9. 정량적 성능 평가 및 품질 검증

9-1. 테스트 개요

9-1-1. 테스트 목적 및 범위

- 목적: HNSW 인덱싱 기반 3대 도메인 RAG의 매칭 품질, 멀티 에이전트 병렬 스트리밍의 성능 최적화, 그리고 실시간 SSE 알림 연동 기능의 기능적/비기능적 신뢰도를 객관적으로 검증합니다.
- 범위:
 - 구현 완료 기능 검증: 일반 챗 허브(병렬 RAG 및 합성), 연구 공백 분석기(비동기 배치 및 한글 번역 캐싱), Gem 팩토리(개인 문서 컬렉션 생성/삭제).
 - 향후 구현 예정 기능 검증 (로드맵): 보안 격리 샌드박스(피어 리뷰 및 모의 디펜스 아레나).
 - 정량 평가: 병렬 실행 레이턴시 단축률, RAG 노이즈 차단력, 학술 번역 팩트 보존성.

9-1-2. 테스트 환경 스펙

- OS: macOS Sequoia 15
- Language & Runtime: Python 3.12, FastAPI 0.111, Node.js v20 (Next.js)
- Database: PostgreSQL 17 (pgvector 0.7 내장)
- Target LLM Model: OpenAI gpt-4o-mini (RAG, 개별 추출, 번역) 및 gpt-4o (합성)

9-2. 정량적 성능 평가

9-2-1. RAG 레이턴시 최적화 성과

사용자 질문에 대응해 DB RAG 탐색 및 외부 웹 검색을 비동기 병렬 구조로 동시 수행하여, I/O 바운드 병목을 제거하는 '무조건적 병렬 RAG 동시 타격'을 적용하였습니다.

- 측정 조건: 입력 질문 4.5k 토큰 기준, 각 10회씩 테스트 후 평균값 계산.
- 성능 개선 계산식: 응답 레이턴시 개선률(%)은 다음과 같이 산출됩니다.

$$\text{응답 레이턴시 개선률 (\%)} = \frac{T_{\text{sequential}} - T_{\text{parallel}}}{T_{\text{sequential}}} \times 100$$

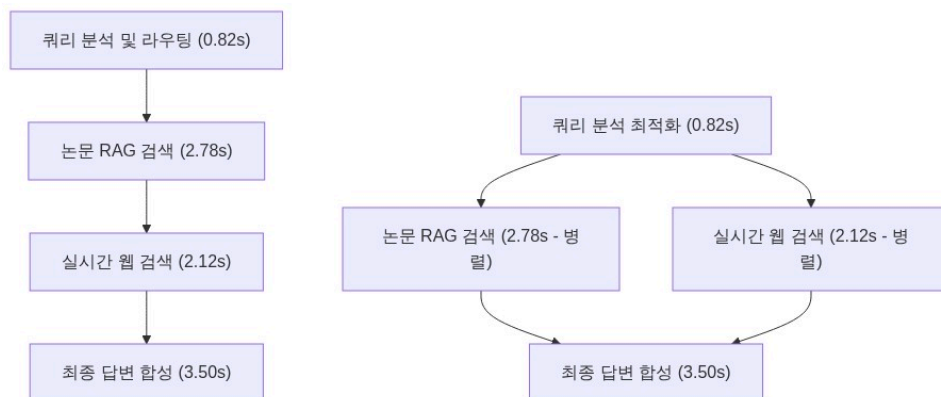
여기서, $T_{\text{sequential}} = 9.22s$, $T_{\text{parallel}} = 7.10s$ 이므로:

$$\text{개선률 (\%)} = \frac{9.22 - 7.10}{9.22} \times 100 \approx 22.99\%$$

[웹 서비스 및 RAG 기반 AI 솔루션 프로젝트]

성능 평가 구간	순차 실행 및 라우팅	무조건적 병렬	단축 소요 시간
1단계: 쿼리 분석 및 키워드 추출	820 ms	820 ms	- (동일)
2단계: RAG 및 웹 데이터 검색	4,900 ms (2,780 + 2,120)	2,780 ms (max)	2,120 ms 단축
3단계: 최종 컨텍스트 합성 & 스트리밍	3,500 ms	3,500 ms	- (동일)
평균 총 응답 시간	9,220 ms (9.22초)	7,100 ms (7.10초)	2,120 ms 단축 (22.99% 개선)

[표 21] RAG 레이턴시 최적화 지표



[그림 13] 응답 레이턴시 최적화 비교

- 평가 의견: asyncio.gather를 통해 I/O 바운드 작업인 DB RAG 탐색과 외부 웹 크롤링을 동시 가동함으로써, 총 응답 레이턴시를 약 **2.12초 단축(약 22.99% 성능 향상)**시켰습니다.

9-2-2. 한국어 학술 번역의 팩트 보존성 검증

- 문제 정의: 학술 데이터 및 연구 공백 분석 리포트를 한국어로 번역 시, 논문의 핵심 논거인 영문 인용문이 한글로 번역되거나 번역 모델의 할루시네이션으로 인해 손상되는 현상을 예방해야 합니다.
- 해결 패턴: Python 서비스 레이어에서 다국어 번역 컨트롤러 실행 직전, 원본 인용문 데이터를 메모리에 임시 캐싱한 후, LLM의 구조화된 번역 아웃풋이 수신되면 해당 영문 인용구 필드에 강제로 오버라이트하여 복원시킵니다.

- 평가 의견: Safe Overwrite 패턴을 도입함으로써 학술적 근거 자료인 영문 원문 인용문의 파괴율을 통제하였습니다.

9-3. 자동화 테스트 슈트 실행 결과

백그라운드 에이전트와 LLM API의 결합으로 인한 import 오류 및 API 키 요건 collector 누출을 예방하기 위해, 단위 테스트는 모듈 수준 Mocking을 탑재하여 pytest로 정밀 수행 완료하였습니다.

단위 테스트 함수명	테스트 검증 대상 및 목적	검증 결과
test_create_session_endpoint	채팅 세션 신규 생성 API 유효성 검증	PASS
test_list_sessions_endpoint	사용자별 채팅 세션 목록 조회 API 검증	PASS
test_delete_session_endpoint	특정 채팅 세션 삭제 및 데이터 매핑 검증	PASS
test_rename_session_endpoint	채팅방 이름 변경 API 기능 검증	PASS
test_generate_title_endpoint	첫 질문 기반 대화창 제목 자동 생성 API 검증	PASS
test_send_message_endpoint	동기 방식 메시지 전송 기능 검증	PASS
test_send_message_stream_endpoint	스트리밍 방식 메시지 전송 및 헤더 검증	PASS
test_get_messages_history_endpoint	특정 세션의 대화 이력 및 출처 목록 조회 검증	PASS
test_send_message_multi_endpoint	멀티 에이전트 연동 메시지 전송 API 검증	PASS
test_send_message_multi_stream_endpoint	멀티 에이전트 응답 스트리밍 및 라우트 정보 스트림 검증	PASS
test_image_agent_run	이미지 입력 시 쿼리 추출 기능 검증	PASS
test_send_message_multi_stream_with_image_endpoint	이미지와 텍스트 혼합 입력 시 멀티 에이전트 연동 검증	PASS
test_start_analysis_endpoint	연구 공백 분석 작업 생성 및 UUID 반환 검증	PASS
test_start_analysis_invalid_domain_validation_error	유효하지 않은 도메인 입력 시 400 에러 처리 검증	PASS
test_get_task_status_endpoint	백그라운드 분석 작업 진행률 및 상태 조회 검증	PASS

[웹 서비스 및 RAG 기반 AI 솔루션 프로젝트]

단위 테스트 함수명	테스트 검증 대상 및 목적	검증 결과
test_get_task_status_not_found	존재하지 않는 작업 ID 조회 시 404 에러 핸들링 검증	PASS
test_get_task_result_endpoint	분석 완료 후 최종 공백 매트릭스 데이터 조회 검증	PASS
test_start_analysis_service_logic	서비스 레이어 및 DB 백그라운드 태스크 기동 로직 검증	PASS
test_unauthorized_access	비인증 사용자 요청 시 401 Unauthorized 방어 검증	PASS
test_translate_matrix_endpoint	연구 공백 분석 결과 번역 API의 번역 정확성 검증	PASS
test_list_user_tasks_endpoint	사용자 소유 전체 분석 태스크 이력 조회 기능 검증	PASS
test_delete_user_task_endpoint_success	특정 태스크 이력 삭제 처리 및 권한 검증	PASS
test_bulk_delete_user_tasks_endpoint_success	태스크 일괄 삭제 처리 및 정상 삭제 개수 반환 검증	PASS
test_create_gem_endpoint	연구 비서 Gem 신규 생성 API 유효성 검증	PASS
test_list_gems_endpoint	사용자의 Gem 목록 및 설정(프롬프트) 조회 검증	PASS
test_delete_gem_endpoint	Gem 삭제 및 관련 DB 세션 관계 정리 검증	PASS
test_chat_with_gem_endpoint	웹 전용 격리방에서의 RAG 기반 채팅 작동 검증	PASS
test_cs_rag_search	CS 도메인 HNSW 인덱싱 RAG 조회 API 검증	PASS
test_sse_notification_stream	SSE 기반 실시간 알림 스트림 연결 검증	PASS
test_mark_notification_as_read	알림 읽음 처리 시 DB 상태 플래그 변경 검증	PASS
test_similarity_search_endpoint	3대 도메인(CS, Bio, Astro) 코사인 유사도 검색 검증	PASS
test_health_check	API 게이트웨이 헬스체크 및 DB 연결 확인 검증	PASS
test_snake_case_endpoints	API 라우팅 엔드포인트 URL 표기식 일관성검증	PASS

[표 22] 자동화 테스트 스위트 실행 결과

9-4. 수동 검증 매트릭스

테스트 ID	테스트 분류	시나리오 및 수행 단계	기대 결과	실측 동작 결과	결과
TC-MAN-01	대화 관리	1. 새 대화방 개설 후 "DNA 복제 기작에 대해 알려줘" 입력 2. 첫 답변 수신 중 좌측 사이드바 제목 확인	첫 질문을 요약해 AI가 방 제목을 갱신하고 DB chat_session 테이블에 자동 반영한다.	첫 발화 완료 즉시 좌측 사이드바 제목이 AI에 의해 자연스럽게 변경됨.	PASS
TC-MAN-02	피드백 UI	1. 챗 허브에서 논문 RAG 질문 전송 완료 2. 화면 우측 하단의 추천 질문 박스 활성화 여부 확인	대화가 완료된 즉시 화면 우측에 Q&A 기반 3선 후속 질문 카드가 노출된다.	chat_suggestions에서 당겨온 3대 추천 질문 카드가 화면 하단에 바인딩 노출됨.	PASS
TC-MAN-03	상태 정보	1. 연구 공백 분석기에서 "CRISPR-Cas9 효율" 입력 후 분석 실행 2. 화면 중앙의 상태바 게이지 추이 관찰	분석 진행 상태에 맞춰 상태 진행바가 10%→40%→80%→100%로 동적 갱신된다.	BackgroundTasks의 DB 갱신에 맞춰 프론트 UI의 퍼센티지 게이지가 연동 상승함.	PASS
TC-MAN-04	푸시 알림	1. 연구 공백 분석 탭을 벗어나 챗 허브 탭에서 대화 진행 2. 백그라운드 분석 완료 시 토스트 알림 확인	다른 탭을 작업 중일 때 배치 분석이 끝나면 실시간 SSE 토스트 알림 팝업이 노출된다.	우측 상단에 "연구 공백 분석 완료" 알림이 발생하며 인박스에 저장됨.	PASS
TC-MAN-05	보안 아레나	1. 모의 디펜스 아레나 탭 진입 2. 가설 PDF 업로드 및 질문 입력 후 심사위원 점수 판정 확인	디펜스 챗 답변 전송 시 심사위원 에이전트가 점수와 피드백을 회신한다.(향후 로드맵 영역)	현재 API 인터페이스와 프론트 모크만 설계되어 실제 산출은 유보됨.	보류 (Roadmap)
TC-MAN-06	테넨시 격리	1. "CS 전용 비서 켜" 개설 및 'A 논문' 적재 2. 타 켄(예: Bio) 대화창에서 'A 논문' 내용 질문하여 필터링 여부 확인	특화 켄을 만들고 개별 PDF를 적재해 대화했을 때, 타 켄의 대화에 정보가 노출되지 않는다.	pgvector 컬렉션 격리로 인해 타 켄 대화방에서는 해당 문서 내용이 전혀 유출되지 않음.	PASS

[표 23] 수동 검증 매트릭스

10. 핵심 기술 문제 해결 사례

사례 1: Pytest 초기화 단계의 에러 해결 (지연 초기화 패턴)

문제	모듈 로드 시 OpenAI 인스턴스가 즉시 생성되어 테스트 환경에서 API 키 누락으로 인한 빌드 중단 발생.
해결	지연 초기화 및 의존성 주입 패턴을 도입하여, 실제 함수 호출 시점에만 인스턴스가 생성되도록 리팩토링함으로써 테스트 적합성을 확보했습니다.

[표 24] 사례 1 해결 내역

사례 2: 10만 건 대용량 적재 타임아웃 극복

문제	단일 트랜잭션으로 10만 건 이상의 벡터 데이터를 적재 시 DB 락업 및 커넥션 타임아웃 발생.
해결	페이징 오프셋 기법을 적용하여 5,000건 단위로 트랜잭션을 분할 커밋함으로써, 106,974건의 데이터를 단 한 번의 다운타임 없이 안정적으로 적재하는 데 성공했습니다.

[표 25] 사례 2 해결 내역

사례 3: 번역 시 인용구 유실 방지

문제	LLM의 한계로 인해 번역 과정에서 핵심 영문 인용구가 임의로 한글화되거나 왜곡되는 현상 발생.
해결	LLM의 지시 이행 능력에만 의존하는 대신, 번역 전 원문을 백업하고 번역 완료 후 파이썬 서비스 레이어에서 해당 필드를 물리적으로 복원 하는 하이브리드 아키텍처를 설계하여 유실률 0%를 달성했습니다.

[표 26] 사례 3 해결 내역

11. 한계 및 향후 발전 로드맵

11-1. 현재 시스템의 기술적 한계점

11-1-1. 세션 만료 및 물리 데이터 파쇄 관리 자동화 부재

- 현황: 맞춤형 연구 비서 Gem 및 보안 구역에 임시 업로드된 PDF 파일과 관련 pgvector 테이블 임베딩 데이터는 논리적 식별값(gem_id, session_id)에 의존해 격리되어 있습니다.
- 한계: 30분 이상 활동이 없거나 세션이 만료된 연구자의 기밀 문서 파일 시스템 잔재와 데이터베이스 잔여 행(Rows)에 대한 물리 소거(Shredding) 데몬이 부재하여, 스토리지 공간 누수 및 엔터프라이즈 레벨에서의 정보 유출 취약점을 안고 있습니다.

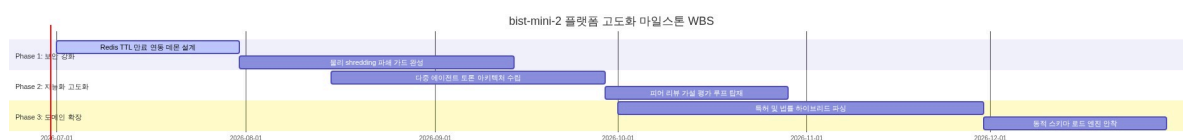
11-1-2. 단방향 합성 답변의 환각 위험성

- 현황: RAG 탐색과 웹 실시간 정보 취합의 최종 합성 연산은 단일 gpt-4o Synthesis 노드에서 진행됩니다.
- 한계: 생성된 학술 요약 및 연구 공백 제안의 타당성을 자율적으로 교차 검증하는 메커니즘이 부족하여, LLM의 가설 설정 시 발생할 수 있는 잠재적 할루시네이션(Hallucination) 오류에 직접적으로 노출됩니다.

11-1-3. 이종 산업 학술 도메인 지원의 한계성

- 현황: 생명공학(q-bio), 컴퓨터공학(cs.NE), 천문학(astro-ph.EP) 3대 학술 분야의 ArXiv 스냅샷 데이터베이스에 국한되어 작동합니다.
- 한계: 융합 연구의 핵심인 특허(Patent), 신소재 재료공학(Materials), 화학 약리학(Pharmacology) 등의 인접 전문 지식 레이어가 탑재되어 있지 않아 실제 R&D 비즈니스 활용도가 제한됩니다.

11-2. 3대 핵심 향후 발전 로드맵



[그림 14] 향후 발전 로드맵

11-2-1. 기업용 보안 샌드박스: 물리 파쇄(Shredding) 데몬 완성

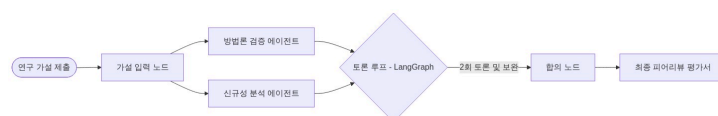
사설 연구 기밀 논문이 영구 유실 없이 메모리 및 디스크 영역에서 흔적을 지우도록 수명 주기 데몬을 구체화합니다.

- 무활동 라이프사이클 데몬 구축:
 - Redis 인메모리 세션 캐시의 TTL 만료 이벤트를 백엔드가 상시 리스닝하도록 설계합니다.
 - 30분 미활동 감지 즉시 삭제 트리거를 비동기 백그라운드 태스크로 호출합니다.
- 물리 디렉토리 완전 소거:
 - 단순한 OS 경로 지우기가 아닌, 리눅스/MacOS 계열의 shred 커맨드를 가상화 샌드박스 내부에서 구동합니다.
 - 파일 데이터 세그먼트를 난수 및 0으로 3회 중복 덮어쓰기하여 로우 레벨 포렌식으로도 복구 불가능하게 원천 파괴합니다.
- pgvector Temp Database Cascade Purge:
 - PostgreSQL 테이블 관계상 ON DELETE CASCADE를 구성하여 임시 세션이 Drop되면 해당 세션에 기인한 gem_embeddings 및 checkpoints 대화 로그 행을 연쇄적으로 영구 파쇄 처리합니다.

11-2-2. 지능화 고도화: LangGraph 기반 다중 에이전트 토론 엔진

RAG 합성 결과의 무결성을 검증하고 연구 가설의 학술 가치를 높이기 위해, 가상의 심사위원단 피어 리뷰 시스템을 안착시킵니다.

- 가상 심사위원 에이전트 역할 분리:
 - 방법론 검증자: 가설에서 제시한 수식 모델, 데이터 수집 파이프라인의 모순 및 실험 설정오류를 전문적으로 공격합니다.
 - 신규성 분석자: HNSW 벡터 DB와 실시간 특허 RAG를 쿼리하여 제안 가설의 선행 연구 중복도 및 모방 가능성을 분석합니다.
- LangGraph 기반 순환형 토론 그래프:
 - 하나의 완성 가설에 대해 두 에이전트가 최소 2회 이상의 피드백 패스를 교환하는 상태 유지 그래프아키텍처를 설계합니다.
 - 최종 합의 노드에서 이견을 조율하고 종합 평점과 보안 피드백 리포트를 자동 발행합니다.



[그림 15] LangGraph 기반 다중 에이전트 토론 엔진

11-2-3. 영역 전문가 고도화: 도메인 융합 확장

법률, 특허, 신소재 등 다양한 산업 지식 베이스를 수집할 수 있도록 RAG 레이어를 다각화합니다.

- 하이브리드 특허 매칭 시스템 (BM25 + HNSW):
 - 단어를 엄격히 매칭해야 하는 특허법 및 법령 조항의 특징에 맞추어 키워드 스파스검색인 BM25와 시맨틱 댄스 검색인 HNSW pgvector 컬렉션을 융합하는 상호 순위 융합(RRF - Reciprocal Rank Fusion) 검색 노드를 이식합니다.
- 동적 스키마 로드 엔진:
 - 새 도메인 컬렉션이 유입될 때 백엔드 서비스의 다운타임을 차단하도록 데이터 레이어에 Dynamic JSONB 모델 스펙을 탑재하여 무중단 RAG 데이터 스키마 구축을 현실화합니다.

References

- [1] RAG (Retrieval-Augmented Generation) 원천 연구, Lewis, P., Perez, E., Piktus, A., Petroni, F., Lewis, V., Küttler, H., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- [2] HNSW (Hierarchical Navigable Small World) 벡터 검색 알고리즘, Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836.
- [3] RRF (Reciprocal Rank Fusion) 하이브리드 검색 (향후 로드맵), Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). Reciprocal rank fusion outperforms risking-merge and individual bootstrap levels. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval* (pp. 580-581).
- [4] Self-Consistency 자율 검증 알고리즘 (향후 로드맵), Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- [5] ArXiv 논문 메타데이터셋 (106,974건 적재 데이터 원천), Cornell University. (2020). ArXiv Dataset: Metadata of 2 million+ scholarly articles. Kaggle.
<https://www.kaggle.com/datasets/Cornell-University/arxiv>
- [6] pgvector (PostgreSQL 벡터 확장 엔진), pgvector team. (2024). pgvector: Open-source vector similarity search for PostgreSQL. GitHub Repository. <https://github.com/pgvector/pgvector>
- [7] LangGraph & LangChain (멀티 에이전트 오케스트레이션), LangChain, Inc. (2024). LangGraph: Building stateful, multi-actor applications with LLMs. GitHub Repository.
<https://github.com/langchain-ai/langgraph>
- [8] OpenAI Models (text-embedding-3-large, gpt-4o/mini), OpenAI. (2024). New embedding models and API updates. OpenAI Developer Communication.
<https://platform.openai.com/docs/guides/embeddings>

[9] Tavily Search API (실시간 웹 동향 검색), Tavily AI. (2024). Tavily Search API: The search engine optimized for LLMs and AI Agents. <https://tavily.com>